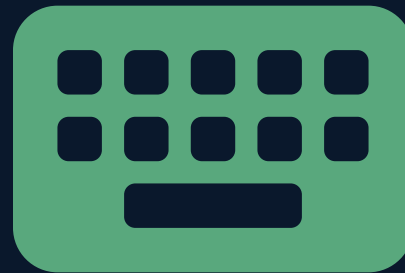


PORTADA

Técnicas de Programación

Unidad 2 — Paradigma Objetual: Entrada y Salida de Información

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



APERTURA

Todos tus programas, hasta hoy, corren siempre exactamente igual. Un programa útil tiene que escuchar a quien lo usa.



TRANSICIÓN

De valores fijos a valores que llegan del usuario

Hasta ahora, tus programas usaban valores escritos directamente en el código (`int promedio = 85;`). Eso es útil para practicar, pero un programa real necesita recibir datos **en el momento en que se ejecuta** — un nombre, una edad, un monto — y luego mostrar resultados de forma clara. Java resuelve esto con la clase `Scanner` para la entrada, y con `System.out` para la salida.

CONCEPTO

La clase **Scanner** : leer desde el teclado

`Scanner` vive en el paquete `java.util` y se conecta a `System.in` (la entrada estándar, normalmente el teclado) para leer lo que el usuario escribe en la consola.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        System.out.print("¿Cuál es tu nombre? ");
        String nombre = teclado.nextLine();
        System.out.println("Hola, " + nombre);
    }
}
```

CONCEPTO

Un método de lectura por cada tipo de dato

Scanner ofrece un método distinto según el tipo de dato que se espera leer — todos bloquean el programa hasta que el usuario presiona Enter.

Método	Lee
nextInt()	Un número entero
nextDouble()	Un número decimal
next()	Una sola palabra (hasta el próximo espacio)
nextLine()	La línea completa, incluidos los espacios

CONTRAEJEMPLO

El bug clásico: mezclar `nextInt()` con `nextLine()`

`nextInt()` lee **solo el número**, dejando el salto de línea (Enter) todavía esperando en el buffer. Si el siguiente `nextLine()` se ejecuta justo después, lee ese salto de línea sobrante en vez de esperar texto nuevo — y la variable queda vacía.

```
Scanner teclado = new Scanner(System.in);
System.out.print("Edad: ");
int edad = teclado.nextInt();           // lee "20", deja el Enter pendiente

System.out.print("Nombre: ");
String nombre = teclado.nextLine();    // lee el Enter sobrante, no el nombre!
System.out.println("Hola " + nombre); // imprime: "Hola " (vacío)
```

CONCEPTO

Cómo evitarlo: consumir el salto de línea pendiente

La solución más simple es agregar un `teclado.nextLine()` extra inmediatamente después de `nextInt()` / `nextDouble()` , solo para "limpiar" el salto de línea que quedó pendiente, antes de leer texto con `nextLine()` .

```
System.out.print("Edad: ");
int edad = teclado.nextInt();
teclado.nextLine();    // consume el Enter pendiente, no guarda su resultado

System.out.print("Nombre: ");
String nombre = teclado.nextLine();    // ahora sí lee el nombre correctamente
System.out.println("Hola " + nombre + ", tienes " + edad + " años.");
```

CONCEPTO

Salida: `print`, `println` y `printf`

Java ofrece tres formas de escribir en consola a través de `System.out` , cada una con un propósito distinto.

Método	Comportamiento
<code>print(x)</code>	Escribe sin salto de línea al final
<code>println(x)</code>	Escribe y agrega un salto de línea al final
<code>printf(formato, valores)</code>	Escribe con formato controlado (decimales, ancho, alineación)

CONCEPTO

Especificadores de formato en printf

`printf` usa una cadena de formato con marcadores `%` que se reemplazan, en orden, por los valores que le siguen.

```
double promedio = 4.6666;  
String nombre = "Ana";  
  
System.out.printf("Estudiante: %s\n", nombre);  
System.out.printf("Promedio: %.2f\n", promedio);    // "Promedio: 4.67"  
System.out.printf("%-10s | %5d\n", "Total", 3);      // ancho y alineación
```

`%s` texto · `%d` entero · `%f` decimal (`%.2f` con 2 decimales) · `%n` salto de línea portable.

TENSIÓN CONCEPTUAL

¿Y si el usuario escribe algo que no es un número?

Si tu programa hace `teclado.nextInt()` y el usuario escribe "veinte" en vez de "20", Java lanza `InputMismatchException` y el programa se detiene abruptamente. Validar esa entrada de forma controlada requiere `try / catch` — un mecanismo de manejo de errores que todavía no hemos visto en el curso, y que se retoma más adelante. Por ahora, basta con saber que esta falla existe y por qué ocurre.

INTERACCIÓN

Un programa que lea y muestre datos

Individual: escriban un programa que use `Scanner` para leer el nombre, la edad y el promedio de un estudiante, mezclando `nextLine()` y `nextDouble()` / `nextInt()` a propósito, y luego muestren los resultados con `printf` usando al menos un `%.2f`.

6 minutos · si el resultado sale vacío, revisen el buffer pendiente del slide anterior

SÍNTESIS

Lo que hay que llevarse de hoy

1. `Scanner` (de `java.util`) conecta el programa con `System.in` para leer datos que escribe el usuario.
2. Cada tipo de dato tiene su propio método de lectura: `nextInt()`, `nextDouble()`, `next()`, `nextLine()`.
3. Mezclar `nextInt()` / `nextDouble()` con `nextLine()` sin consumir el salto de línea pendiente es el bug más común al leer datos.
4. `printf` permite controlar el formato de salida con especificadores como `%s`, `%d`, `%.2f` y `%n`.



CIERRE

Antes de la próxima sesión

- Implementar en el IDE un programa que lea al menos tres datos de tipos distintos con `Scanner` y los muestre con `printf` formateado.
- Revisar el ejercicio del bug de `nextInt()` / `nextLine()` hasta poder explicar exactamente por qué ocurre.

La próxima sesión pasamos del diagrama de clases a la implementación completa en Java.

